
Guião Laboratorial

Aula 02

Sensores, Interrupções
e Programação Visual

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026–2027

Catarina Silva

ESTGA, Universidade de Aveiro

Duração prevista: 2 horas

Índice

Enquadramento	2
Os dois modos de operação utilizados	2
Material e Preparação Prévia	3
Objetivos de Aprendizagem	3
Plano da Sessão	4
Procedimento Inicial: StandardFirmata	5
1 Controlo de Saída Digital a partir do Node-RED	5
2 Leitura de Entrada Digital e Resposta a Eventos	7
3 Aquisição de Grandeza Analógica	9
4 Sensor Digital com Protocolo Próprio (DHT11)	11
5 Sensor de Distância por Ultrassons (HC-SR04)	13
6 Interrupções ao Nível do Microcontrolador	15
7 Painel de Visualização Integrado	18
Referências	18

Enquadramento

O presente guião concretiza, em ambiente laboratorial, os conceitos apresentados na componente teórica das Aulas 01 e 02: a arquitetura de referência de sistemas IoT, a conversão analógico-digital e os mecanismos de resposta a eventos.

A abordagem adotada privilegia **ferramentas de programação visual** (*low-code*). Sempre que a tecnologia o permite, a lógica da aplicação é construída no Node-RED, através da ligação de blocos funcionais, sem redação de código. O código-fonte apenas é utilizado nos casos em que os sensores recorrem a protocolos temporizados que o firmware genérico não suporta; nesses casos, os *sketches* são fornecidos prontos a carregar, não cabendo ao estudante escrevê-los.

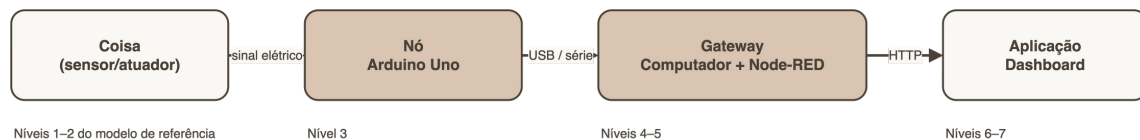
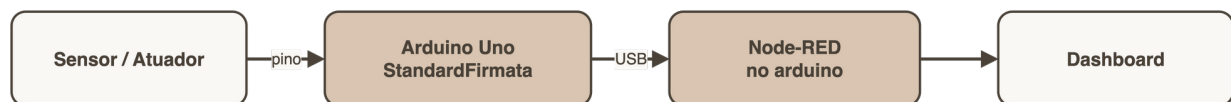


Figura 1: Posicionamento dos exercícios na cadeia IoT e no modelo de referência da Aula 01

Os dois modos de operação utilizados

Ao longo da sessão são utilizados dois modos distintos de articulação entre o Arduino e o Node-RED, cuja distinção é essencial para compreender os exercícios.

Modo A — Sem código no Arduino (StandardFirmata)



Modo B — Sketch dedicado + porta série



Figura 2: Modo A (Firmata, sem código) e Modo B (sketch dedicado com comunicação série)

Modo A — Firmata

O Arduino executa o firmware genérico **StandardFirmata**, carregado uma única vez no início da sessão. Este firmware expõe os pinos da placa ao computador, permitindo que o Node-RED os leia e escreva diretamente. Não é escrito qualquer código. Aplica-se aos exercícios 1, 2 e 3.

Modo B — Sketch dedicado com comunicação série

Os sensores DHT11 e HC-SR04 utilizam protocolos com requisitos temporais estritos, não suportados pelo **StandardFirmata**. Nestes casos é carregado um *sketch* específico, fornecido com o guião, que efetua a leitura e envia o resultado pela porta série. O tratamento e a visualização dos dados continuam a ser realizados no Node-RED. Aplica-se aos exercícios 4 e 5.

Material e Preparação Prévia

As sessões laboratoriais decorrem em computadores pessoais dos estudantes. A instalação do software deve, por esse motivo, estar concluída antes do início da aula.

Equipamento (disponibilizado no laboratório)

- Arduino Uno e respetivo cabo USB
- *Breadboard* e fios de ligação
- Botão de pressão e resistência de 10 k Ω
- Sensor de humidade do solo (higrómetro resistivo)
- Sensor DHT11 (temperatura e humidade)
- Sensor HC-SR04 (distância por ultrassons)

Software a instalar previamente

1. **Arduino IDE**, a partir de arduino.cc/en/software. É utilizado exclusivamente para carregar firmware na placa.
2. **Node.js**, versão LTS, a partir de nodejs.org.
3. **Node-RED**, através do comando `npm install -g --unsafe-perm node-red`.
4. **Pacotes adicionais do Node-RED**, instalados em `≡ > Manage palette > Install: node-red-node-arduino, node-red-node-serialport e @flowfuse/node-red-dashboard`.
5. **Bibliotecas Arduino**, instaladas em `Tools > Manage Libraries`, procurando pelo nome: `TimerOne`, usada no exercício de interrupção por software, e `DHTlib`, de Rob Tillaart, usada na leitura do sensor DHT11. Caso alguma não conste do gestor, pode ser obtida no repositório indicado nas referências e importada em `Sketch > Include Library > Add .ZIP Library`.
6. Em sistemas Windows, e caso a placa não seja reconhecida, deve ser instalado o controlador USB-série CH340.

Nota sobre as bibliotecas

As duas bibliotecas não são distribuídas com o Arduino IDE nem com este guião. São projetos de terceiros, de utilização livre, e devem ser instaladas pelo próprio estudante antes da sessão. A instalação demora menos de um minuto por biblioteca e não requer registo.

Verificação de funcionamento

Executar `node-red` num terminal e aceder a `http://127.0.0.1:1880`. Deve ser apresentado o editor de fluxos. A paleta de nós, à esquerda, deve incluir as secções `arduino`, `dashboard` e o nó `serial in`.

Objetivos de Aprendizagem

No final da sessão, o estudante deverá ser capaz de:

- Estabelecer a ligação física entre sensores e atuadores e uma plataforma de microcontrolador, identificando corretamente alimentação, massa e linhas de sinal;

- Construir fluxos de aquisição, tratamento e visualização de dados em Node-RED, sem recurso a programação textual;
- Distinguir sensores analógicos de sensores digitais e relacionar a leitura obtida com a resolução do conversor analógico-digital;
- Distinguir os paradigmas de sondagem periódica e de interrupção, identificando as respetivas manifestações ao nível do fluxo visual e ao nível do microcontrolador;
- Integrar múltiplas fontes de dados num painel de visualização único.

Plano da Sessão

N.º	Exercício	Modo	Duração
1	Controlo de saída digital a partir do Node-RED	A	15 min
2	Leitura de entrada digital e resposta a eventos	A	20 min
3	Aquisição de grandeza analógica (humidade do solo)	A	15 min
4	Sensor digital com protocolo próprio (DHT11)	B	20 min
5	Sensor de distância por ultrassons (HC-SR04)	B	20 min
6	Interrupções ao nível do microcontrolador	—	20 min
7	Painel de visualização integrado	A + B	10 min

Organização do trabalho

Recomenda-se a constituição de grupos de dois estudantes, preferencialmente com percursos formativos distintos. Sugere-se que a montagem dos circuitos e a construção dos fluxos sejam alternadas entre os elementos do grupo a meio da sessão, de modo a assegurar que ambos contactam com as duas dimensões do trabalho.

Procedimento Inicial: Carregamento do StandardFirmata

Este procedimento é realizado uma única vez e habilita os exercícios 1 a 3.

1. Ligar o Arduino Uno ao computador através do cabo USB.
2. No Arduino IDE, selecionar **Tools > Board > Arduino Uno** e, em **Tools > Port**, a porta correspondente à placa. **Registrar a designação desta porta**, necessária em todos os exercícios seguintes.
3. Abrir **File > Examples > Firmata > StandardFirmata**.
4. Carregar o *sketch* na placa através de **Sketch > Upload**.
5. Encerrar o Arduino IDE, ou pelo menos o *Serial Monitor*. A porta série apenas pode ser utilizada por uma aplicação de cada vez, e será requerida pelo Node-RED.

Nota

A partir deste ponto, e até ao exercício 4, não é carregado qualquer outro programa na placa. Toda a lógica é definida no Node-RED.

1. Controlo de Saída Digital a partir do Node-RED

Modo A — Firmata / 15 min

Objetivo

Acionar o LED integrado na placa, associado ao pino D13, através de um interruptor apresentado num painel web, sem redação de código.

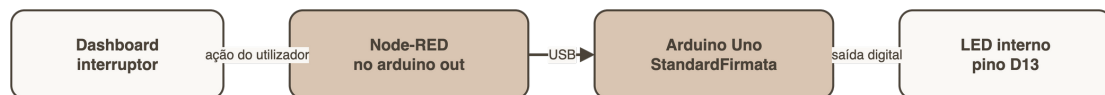


Figura 3: Arquitetura do exercício 1

Ligações

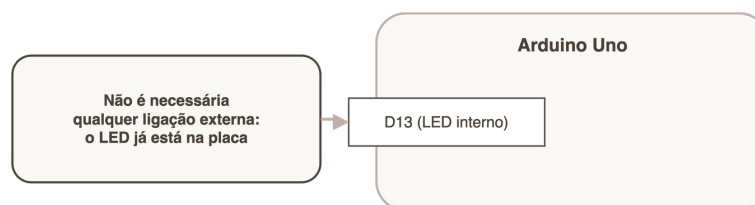


Figura 4: Ligações do exercício 1

Construção do fluxo

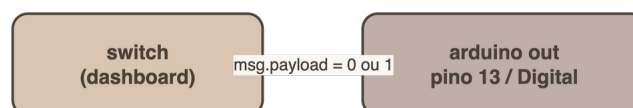


Figura 5: Fluxo Node-RED do exercício 1

1. Arrastar para a área de trabalho um nó **ui-switch** da secção *dashboard*.
2. Abrir o nó com duplo clique. O campo **Group** pode ficar vazio; ao fazer **Deploy** pela primeira vez, o Dashboard cria automaticamente a página e o grupo necessários.
3. Ainda no mesmo diálogo, localizar as linhas **On Payload** e **Off Payload**. Cada uma tem, do lado esquerdo, um pequeno seletor de tipo. Abrir esse seletor, escolher **number** e escrever 1 na primeira linha e 0 na segunda.
4. Arrastar um nó **arduino out** da secção *arduino*.
5. Editar o nó: no campo **Arduino Port** indicar a porta registada no procedimento inicial; em **Pin** indicar 13; em **Type** seleccionar **Digital (0/1)**.
6. Ligar a saída do nó **ui-switch** à entrada do nó **arduino out**.
7. Premir **Deploy**.

Se os campos não aparecerem onde se indica

Existem duas versões do painel do Node-RED, com nós e endereços diferentes. Este guião usa a versão 2, cujo pacote se chama **@flowfuse/node-red-dashboard** e cujos nós têm todos o prefixo **ui-**. A versão 1, **node-red-dashboard**, foi descontinuada em junho de 2024, tem nós sem prefixo e serve o painel em **/ui** em vez de **/dashboard**. Se a paleta mostrar um nó chamado **switch** na secção *dashboard*, em vez de **ui-switch**, está instalada a versão antiga. Nesse caso, desinstalar **node-red-dashboard** em **Manage palette** e instalar **@flowfuse/node-red-dashboard**.

Verificação

Aceder a <http://127.0.0.1:1880/dashboard>. O acionamento do interruptor deve acender e apagar o LED da placa. Confirmar igualmente que o nó **arduino out** apresenta o estado **connected** no editor.

Trabalho complementar

Substituir o nó **ui-switch** por um nó **inject** configurado para repetir a cada segundo, seguido de um nó **function** que alterne o valor enviado. Obtém-se assim, sem código na placa, o comportamento intermitente tradicionalmente programado com **delay()**.

2. Leitura de Entrada Digital e Resposta a Eventos

Modo A — Firmata / 20 min

Objetivo

Detetar a atuação de um botão de pressão e comandar o LED em função do seu estado, observando que o fluxo apenas é executado quando ocorre uma alteração no pino.

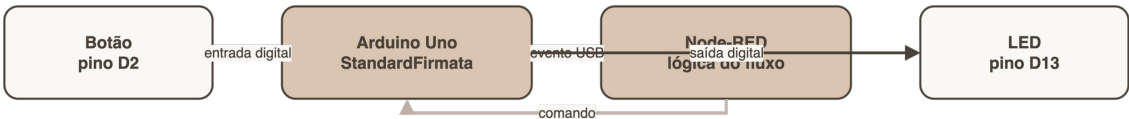


Figura 6: Arquitetura do exercício 2

Ligações

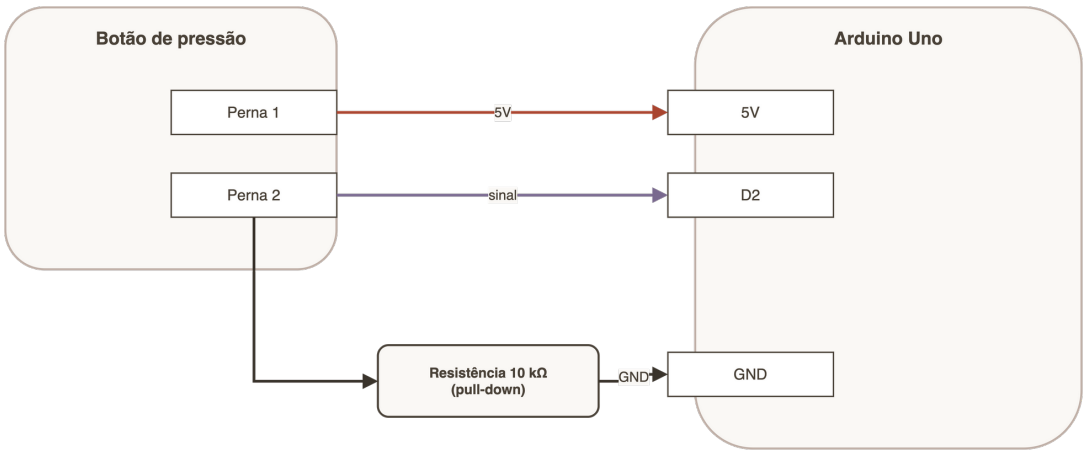


Figura 7: Ligações do exercício 2

Botão	Arduino	Função
Perna 1	5V	Alimentação
Perna 2	D2	Linha de sinal
Perna 2	GND, através da resistência de 10 kΩ	<i>Pull-down</i> : garante o nível lógico 0 com o botão em repouso

Montagem

As ligações devem ser efetuadas com a placa desligada do cabo USB. A resistência de *pull-down* é indispensável: sem ela, a entrada fica em estado flutuante e o valor lido oscila de forma imprevisível.

Construção do fluxo

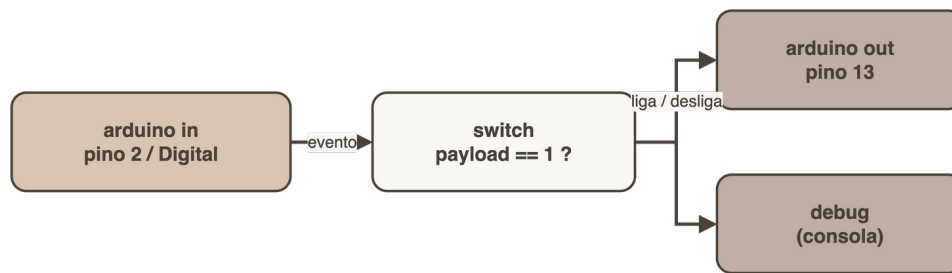


Figura 8: Fluxo Node-RED do exercício 2

1. Arrastar um nó **arduino in**. Configurar a porta, o pino 2 e o tipo **Digital** (0/1).
2. Arrastar um nó **switch** da secção *function*. Definir a regra `== 1` na primeira saída e `otherwise` na segunda.
3. Arrastar um nó **arduino out**, configurado para o pino 13, tipo **Digital** (0/1).
4. Arrastar um nó **debug** e ligá-lo igualmente à saída do nó **arduino in**, de modo a observar as mensagens na consola lateral.
5. Estabelecer as ligações e premir **Deploy**.

Verificação

Ao premir o botão, o LED deve acender; ao libertá-lo, deve apagar-se. Na consola de *debug* observa-se a emissão de mensagens apenas nos instantes de transição, e não de forma contínua.

Observação conceptual

O nó **arduino in** não é executado ciclicamente pelo utilizador: emite uma mensagem quando o valor do pino se altera. Trata-se, ao nível do fluxo visual, do mesmo princípio de funcionamento orientado a eventos que, ao nível do microcontrolador, é concretizado por uma rotina de serviço de interrupção. O exercício 6 estabelece esta correspondência.

3. Aquisição de Grandeza Analógica

Modo A — Firmata / 15 min

Objetivo

Adquirir a leitura do sensor de humidade do solo através do conversor analógico-digital do Arduino e representá-la graficamente, relacionando o valor obtido com a resolução de 10 bits discutida na componente teórica.

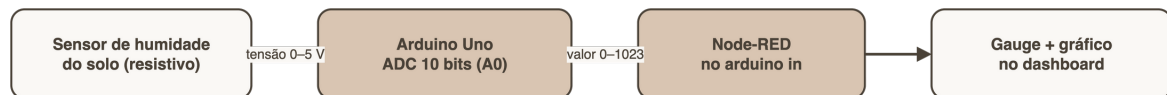


Figura 9: Arquitetura do exercício 3

Ligações

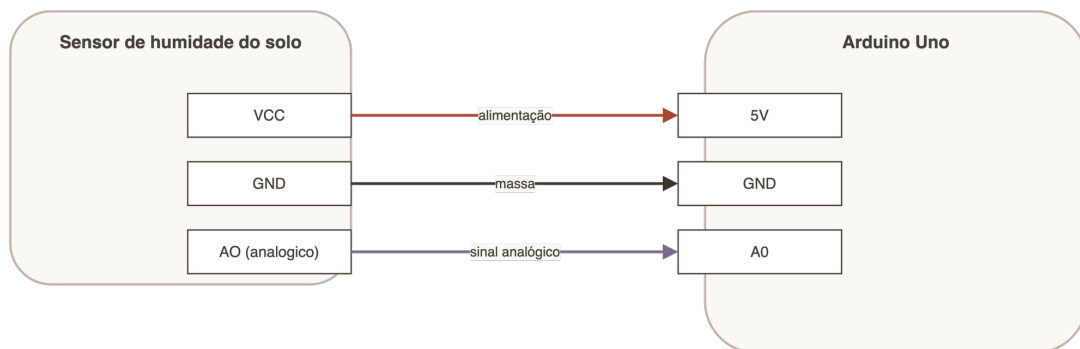


Figura 10: Ligações do exercício 3

Sensor	Arduino	Função
VCC	5V	Alimentação
GND	GND	Massa
AO	A0	Saída analógica do sensor

Construção do fluxo

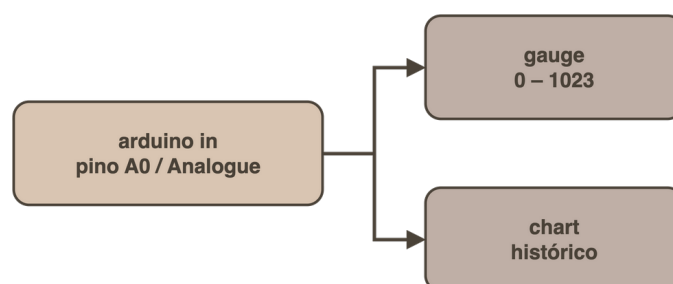


Figura 11: Fluxo Node-RED do exercício 3

1. Arrastar um nó `arduino in`. Configurar a porta, o pino A0 e o tipo Analogue (0-1023).

2. Arrastar um nó **ui-gauge** da secção *dashboard*. Definir o intervalo de 0 a 1023 e atribuir-lhe a designação *Humidade do solo*.
3. Arrastar um nó **ui-chart** da mesma secção, mantendo a configuração predefinida.
4. Ligar ambos à saída do nó **arduino in** e premir **Deploy**.

Verificação

No painel, o valor apresentado deve variar consoante o grau de humidade a que as hastes do sensor estão sujeitas. Registar o valor obtido com o sensor ao ar e com as hastes humedecidas.

Relação com a componente teórica

O valor apresentado situa-se entre 0 e 1023 por resultar de uma conversão com resolução de 10 bits ($2^{10} = 1024$ níveis discretos), aplicada a uma tensão compreendida entre 0 V e a tensão de referência de 5 V.

Trabalho complementar

Inserir um nó **range** entre a aquisição e o **ui-gauge**, convertendo a escala de 0–1023 para uma percentagem de 0 a 100. Trata-se de uma operação de escalamento realizada integralmente por configuração, sem programação.

4. Sensor Digital com Protocolo Próprio (DHT11)

Modo B — sketch fornecido / 20 min

Objetivo

Adquirir temperatura e humidade relativa a partir do sensor DHT11 e apresentá-las em dois indicadores distintos, compreendendo a razão pela qual este sensor exige firmware dedicado.

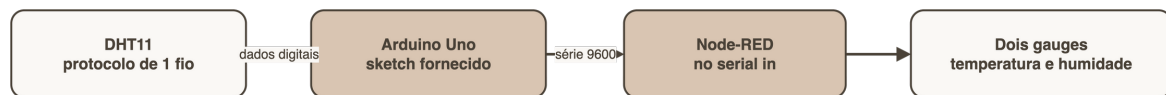


Figura 12: Arquitetura do exercício 4

Ligações

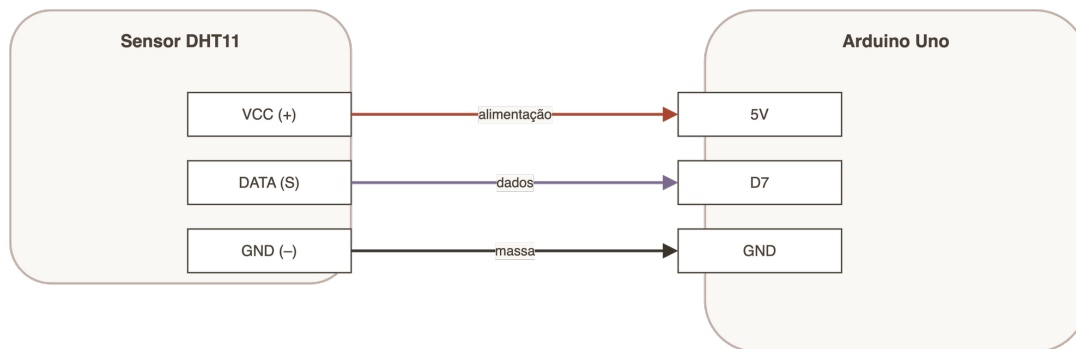


Figura 13: Ligações do exercício 4

DHT11	Arduino	Função
VCC (+)	5V	Alimentação
DATA (S)	D7	Linha bidirecional de dados
GND (-)	GND	Massa

Carregamento do firmware

O DHT11 transmite os dados através de uma única linha, recorrendo a uma codificação temporal com resolução de microssegundos. Este protocolo não é suportado pelo **StandardFirmata**, pelo que é necessário carregar um *sketch* dedicado.

1. Confirmar que a biblioteca **DHTlib** está instalada, conforme indicado na secção de preparação prévia.
2. Abrir o ficheiro **dht11_serie.ino**, fornecido com este guião, e carregá-lo na placa.
3. Encerrar o *Serial Monitor* antes de regressar ao Node-RED.

```

#include <dht.h>
dht DHT;
#define DHT11_PIN 7

void setup() { Serial.begin(9600); }
  
```

```
void loop() {
  DHT.read11(DHT11_PIN);
  Serial.print(DHT.temperature);
  Serial.print(",");
  Serial.println(DHT.humidity);
  delay(2000);
}
```

Cada linha enviada tem o formato `temperatura,humidade`, adequado a tratamento posterior no Node-RED.

Construção do fluxo

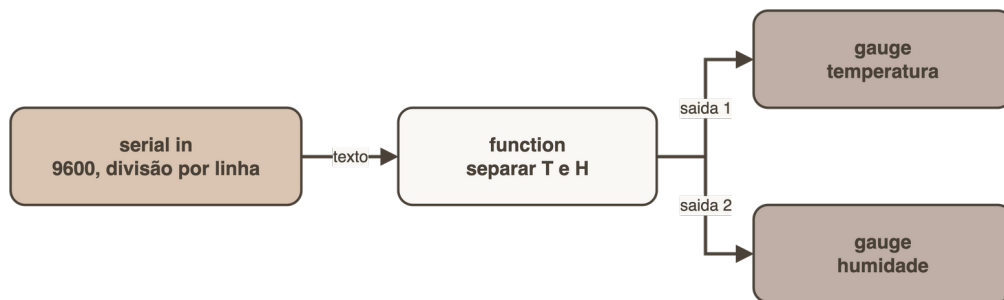


Figura 14: Fluxo Node-RED do exercício 4

1. Arrastar um nó **serial in**. Configurar a porta registada anteriormente, o débito 9600 e a divisão da entrada pelo carácter de fim de linha.
2. Arrastar um nó **function** com duas saídas, contendo:

```
var v = msg.payload.split(",");
return [ { payload: parseFloat(v[0]) },
        { payload: parseFloat(v[1]) } ];
```

3. Associar a primeira saída a um nó **ui-gauge** designado *Temperatura*, com intervalo de 0 a 50.
4. Associar a segunda saída a um nó **ui-gauge** designado *Humidade relativa*, com intervalo de 0 a 100.
5. Premir **Deploy**.

Verificação

Os dois indicadores devem atualizar-se a cada dois segundos. A aproximação da mão ao sensor deve produzir uma variação perceptível em ambas as grandezas.

5. Sensor de Distância por Ultrassons (HC-SR04)

Modo B — sketch fornecido / 20 min

Objetivo

Determinar a distância a um obstáculo por medição do tempo de propagação de um impulso ultrassônico e representar a sua evolução temporal.

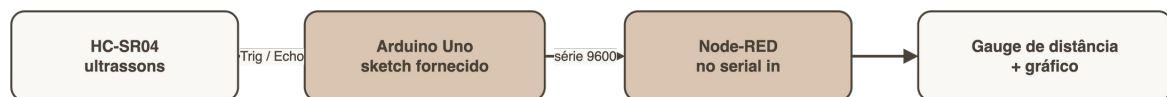


Figura 15: Arquitetura do exercício 5

Ligações

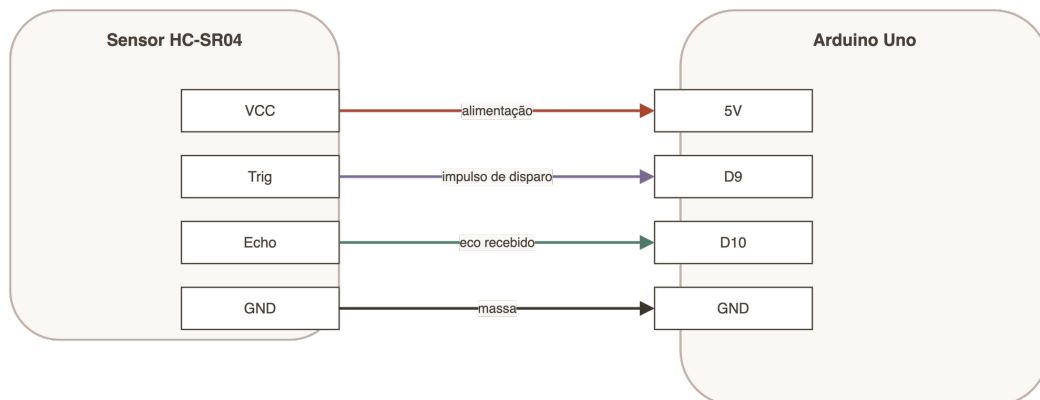


Figura 16: Ligações do exercício 5

HC-SR04	Arduino	Função
VCC	5V	Alimentação
Trig	D9	Impulso de disparo emitido pela placa
Echo	D10	Impulso de retorno, de duração proporcional à distância
GND	GND	Massa

Carregamento do firmware

Carregar o ficheiro `hcsr04_serie.ino`, fornecido com este guião. A medição exige temporização ao microssegundo, igualmente incompatível com o `StandardFirmata`.

```

const int trigPin = 9;
const int echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {

```

```
digitalWrite(trigPin, LOW); delayMicroseconds(2);
digitalWrite(trigPin, HIGH); delayMicroseconds(10);
digitalWrite(trigPin, LOW);

long duracao = pulseIn(echoPin, HIGH);
Serial.println(duracao * 0.034 / 2);
delay(500);
}
```

Fundamento da medição

A função `pulseIn` mede a duração do impulso recebido na linha *Echo*, que corresponde ao tempo de ida e volta do sinal sonoro. Multiplicando essa duração pela velocidade do som ($\approx 0,034 \text{ cm}/\mu\text{s}$) e dividindo por dois, obtém-se a distância ao obstáculo.

Construção do fluxo

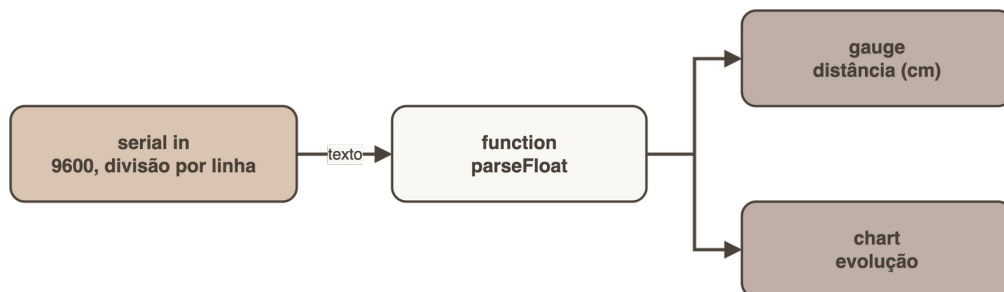


Figura 17: Fluxo Node-RED do exercício 5

1. Arrastar um nó `serial in`, configurado tal como no exercício anterior.
2. Arrastar um nó `function` contendo `msg.payload = parseFloat(msg.payload); return msg;`
3. Associar um nó `ui-gauge` designado *Distância*, com intervalo de 0 a 200, e um nó `ui-chart` para registo da evolução temporal.
4. Premir `Deploy`.

Verificação

Deslocando um obstáculo perante o sensor, o indicador deve acompanhar a variação e o gráfico deve registar o respetivo histórico.

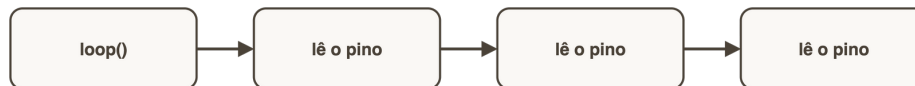
6. Interrupções ao Nível do Microcontrolador

Sketches fornecidos / 20 min

Objetivo

Estabelecer a correspondência entre o funcionamento orientado a eventos observado nos fluxos Node-RED e o mecanismo de interrupção implementado no microcontrolador, distinguindo-o da sondagem periódica.

Sondagem periódica (polling)



O processador verifica repetidamente, mesmo quando não ocorre qualquer evento

Interrupção



O processador só é desviado no instante em que o evento ocorre

Figura 18: Sondagem periódica e interrupção

Parte A — O paradigma observado no fluxo visual

Antes de recorrer a código, importa observar os dois paradigmas no próprio Node-RED.



Figura 19: Fluxo de sondagem periódica em Node-RED

1. Construir o fluxo acima, com um nó **inject** configurado para repetir a cada 3 segundos, um nó **function** que incremente um contador e um nó **ui-text** do painel.
2. Comparar o seu comportamento com o do fluxo do exercício 2. No fluxo com o nó **inject**, as mensagens são emitidas em permanência, independentemente de existir ou não alteração no ambiente; no fluxo do botão, apenas são emitidas quando ocorre uma transição.

Parte B — Interrupção por hardware

Carregar o *sketch interrupcao_hw.ino*, fornecido com o guião. As ligações são as do exercício 2, que se mantêm inalteradas.

```

const int buttonPin = 2;
const int ledPin = 13;
volatile int buttonState = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
  attachInterrupt(digitalPinToInterrupt(buttonPin), pin_ISR, CHANGE);
}
  
```

```
void loop() { /* permanece vazio */ }

void pin_ISR() {
  buttonState = digitalRead(buttonPin);
  digitalWrite(ledPin, buttonState);
}
```

Elementos a observar

O ciclo `loop()` encontra-se vazio: a totalidade do comportamento reside na rotina `pin_ISR`, invocada automaticamente pelo controlador de interrupções sempre que o pino D2 altera de estado. A variável partilhada é declarada `volatile`, instruindo o compilador a lê-la sempre da memória, uma vez que pode ser modificada fora do fluxo normal de execução.

Parte C — Interrupção por software (temporizador)

Carregar o *sketch* `interrupcao_sw.ino`. Este exercício não requer ligações adicionais.

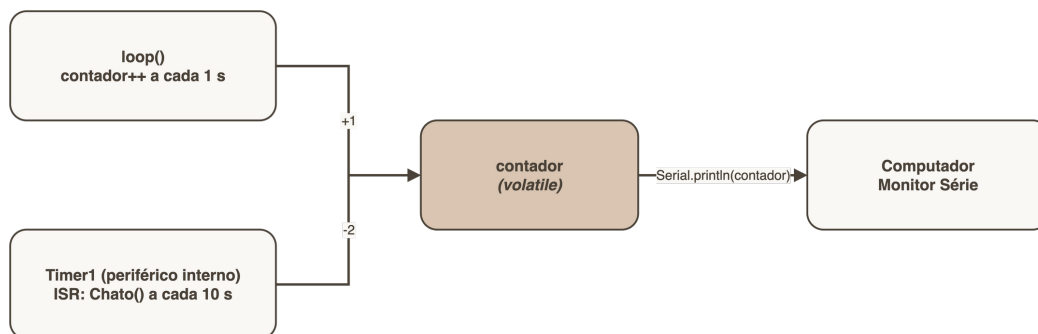


Figura 20: Dois fluxos de execução a atuar sobre a mesma variável

```
#include <TimerOne.h>
volatile int contador = 0;

void subtrai() {
  contador -= 2;
  Serial.print("ISR do temporizador: ");
  Serial.println(contador);
}

void setup() {
  Serial.begin(9600);
  Timer1.initialize(10000000); // 10 segundos, em microssegundos
  Timer1.attachInterrupt(subtrai);
}

void loop() {
  delay(1000);
  contador++;
  Serial.print("Ciclo principal: ");
  Serial.println(contador);
}
```

Questões para discussão

1. Qual dos paradigmas — sondagem ou interrupção — apresenta menor consumo de recursos de processamento? Justificar.

2. Em que circunstâncias poderia a sondagem periódica não detetar um evento de curta duração?
3. Que problema surgiria caso a variável `contador` não fosse declarada `volatile`?
4. Por que razão se recomenda que uma rotina de serviço de interrupção seja tão breve quanto possível, e que não contenha chamadas a `delay()`?

7. Painel de Visualização Integrado

Modos A e B | 10 min

Objetivo

Reunir num único painel as leituras provenientes das diferentes fontes utilizadas ao longo da sessão, constituindo uma aplicação de supervisão elementar.

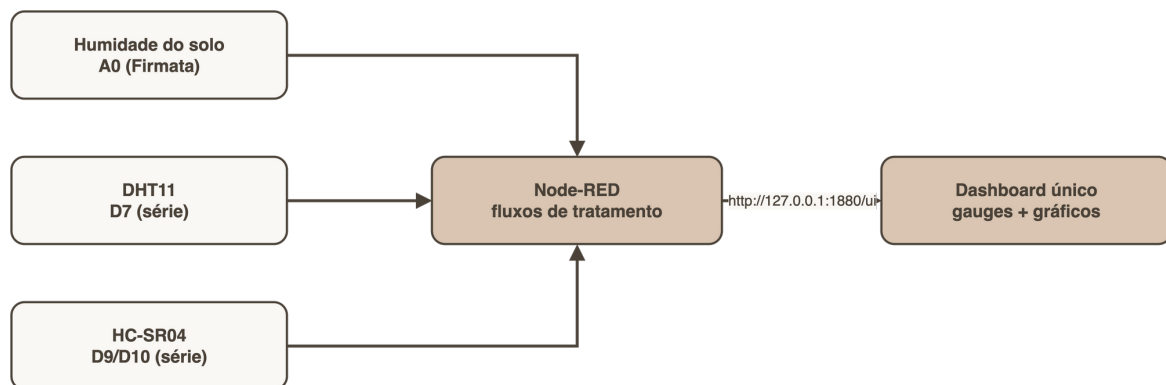


Figura 21: Arquitetura do painel integrado

Procedimento

1. Recarregar o **StandardFirmata** caso se pretenda utilizar simultaneamente a aquisição analógica do exercício 3.
2. Organizar os nós de painel em grupos distintos, através do separador **dashboard** do editor, atribuindo uma designação a cada grupo.
3. Ordenar os grupos de modo a obter uma disposição legível.
4. Premir **Deploy** e aceder a <http://127.0.0.1:1880/dashboard>.

Limitação a considerar

A porta série admite uma única aplicação em simultâneo, e a placa executa apenas um firmware de cada vez. Por conseguinte, não é possível operar em Modo A e em Modo B no mesmo Arduino simultaneamente. Esta limitação é precisamente uma das motivações para as arquiteturas estudadas nas aulas seguintes, nas quais cada dispositivo comunica de forma autónoma através de um protocolo de rede.

Trabalho complementar

Acrescentar um nó **switch** da secção *function* que, quando a distância medida for inferior a um limiar definido, acione o LED da placa. Obtém-se assim uma regra de automação completa, especificada exclusivamente por configuração visual.

Referências

- Documentação do Node-RED: <https://nodered.org/docs/>
- Nó de ligação ao Arduino: <https://flows.nodered.org/node/node-red-node-arduino>
- Nós de comunicação série: <https://flows.nodered.org/node/node-red-node-serialport>

- Nós de painel: <https://dashboard.flowfuse.com>
- Protocolo Firmata: <https://github.com/firmata/protocol>
- Interrupções externas no Arduino: <https://www.arduino.cc/reference/pt/language/functions/external-interrupts/attachinterrupt/>
- Sensor HC-SR04:
<https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/>
- Sensor DHT11:
<https://www.circuitbasics.com/how-to-set-up-the-dht11-humidity-sensor-on-an-arduino/>
- Biblioteca TimerOne, de Paul Stoffregen:
<https://github.com/PaulStoffregen/TimerOne>
- Biblioteca DHTlib, de Rob Tillaart:
<https://github.com/RobTillaart/DHTlib>